```
@SpringBootApplication
@EnableConfigServer                              ❶
public class ConfigServiceApplication {
  public static void main(String[] args) {
    SpringApplication.run(ConfigServiceApplication.class, args);
  }
}
```

❶ Activates the configuration server implementation in the Spring Boot application

That was it on the Java side.

CONFIGURING THE CONFIGURATION SERVER

The next step is configuring the behavior of the config server. Yes, that's right. Even a configuration server needs configuration! First of all, Spring Cloud Config Server runs on an embedded Tomcat server, so you can configure connection timeouts and thread pools as you did for Catalog Service.

You previously initialized a Git repository to host configuration data, so you should now instruct Spring Cloud Config Server where to find it. You can do so in an application.yml file located in the src/main/resources path of your Config Service project (rename the autogenerated application.properties file as application.yml).

The `spring.cloud.config.server.git.uri` property should point to where you defined the config repo. If you followed along with what I did, it will be on GitHub, and the default branch will be called `main`. You can configure which branch the config server should consider by default by setting the `spring.cloud.config.server.git.default-label` property. Remember that, when using Git